



PRODUCT REQUIREMENTS DOCUMENT

MediDesk v3.0

From hospital admin tool to a patient-facing health platform

Prepared for	jhraihaan — this is your build plan
Date	11 September 2026
Builds on	v2.0, branch <code>hms-hardening</code> (8 commits, 95 tests passing)
Stack	Django 6.1 · DRF · React 19 · Tailwind 4 · React Query · Postgres · Redis · Celery · Channels
Scope	4 new features, role-specific interfaces, no AI
Effort	≈ 7–9 weeks solo, in 5 phases

Contents

1	What this is and why
2	What v2.0 already gives you
3	Scope decisions — what's in, what's out, and why
4	Role-specific interfaces (the structural change)
5	Feature A — Smart Queue & Waiting Time
6	Feature B — Digital Medical Records
7	Feature C — Blood Donor Network
8	Feature D — Medicine Reminders
9	Data model changes
10	Technical architecture
11	Non-functional requirements
12	Build order
13	Risks
14	Appendix — dependencies and definition of done

1 What this is and why

v2.0 made the system safe. v3.0 makes it useful to patients.

MediDesk today is a competent hospital administration tool. Staff can manage patients, book appointments, write prescriptions, dispense medicine and take payments. What it cannot do is solve any of the problems a patient in Bangladesh actually walks in with.

This document specifies four features that change that, each chosen because it addresses a real and common frustration, and because it is genuinely finishable with the stack you already know.

The problem	What v3.0 does about it
Long, unpredictable waits. Patients travel far, then sit for hours with no idea when they will be seen, or whether the doctor turned up at all.	A live queue. Your position, an estimated wait in minutes, and the doctor's real availability — updating on screen without a refresh.
Lost paper records. Patients carry envelopes of old reports between hospitals, and lose them.	A personal record vault. Upload prescriptions and test reports, keep them forever, and share a specific report with a specific doctor for a limited time.
Blood emergencies. Finding a matching donor at 2am means phoning everyone you know.	A donor network. Post an urgent request; every compatible, available, nearby donor is notified at once.
Missed medication. Elderly patients forget doses and follow-ups, and nobody notices until they get worse.	Reminders that escalate. Scheduled doses, a one-tap "taken", and a nudge to a family member when doses are repeatedly missed.

The honest summary

Two of these — the queue and the reminders — are mostly assembly work on models you already have. The records vault and the donor network are genuinely new, and the donor network is the only one that carries real safety and privacy weight. Plan accordingly.

2 What v2.0 already gives you

Worth knowing, because it determines how much of v3.0 is new work rather than wiring.

Already built	What v3.0 reuses it for
<code>Appointment</code> with <code>checked_in_at</code> and a status flow	The queue is an ordering over checked-in appointments — the field already exists.
<code>DoctorSchedule</code> and <code>/available-slots/</code>	Doctor availability and booking are done. v3.0 adds the live status on top.
<code>Prescription</code> and <code>PrescriptionMedicine</code>	Reminder schedules are generated from a prescription's existing lines.
<code>Notification</code> model and endpoint	Every alert in v3.0 writes here first; SMS and email are extra channels, not replacements.
<code>Patient.blood_group</code> with real choices	Donor matching uses the enum that already exists.
Record-level authorisation on every endpoint	New endpoints inherit scoping through <code>ScopedViewSet</code> rather than reinventing it.
Audit log, append-only	Record sharing and donor contact both get logged with no new infrastructure.

3 Scope decisions

You proposed eight ideas. Four are in. Here is the reasoning for all eight, including the ones I recommend dropping.

3.1 In scope

#	Feature	Effort	Why it earns a place
1	Smart queue & waiting time	Medium	The single most-felt problem, and you are most of the way there already — <code>checked_in_at</code> , schedules and slot booking all exist.
3	Digital medical records	Medium	Pure value to the patient, no external data needed, and it makes every other feature stickier because the patient now has a reason to keep the account.
4	Blood donor network	Medium	High impact, self-contained, and the matching logic is simple. The hard parts are privacy and consent, not code.
5	Medicine reminders	Low	Cheapest of the four. Celery Beat plus the existing prescription lines gets you most of the way; the family-escalation rule is the genuinely novel bit.

3.2 Out of scope, and why

#	Feature	Why not now
2	Medicine verification	The safety one. Verifying a batch number only means something if you hold the manufacturer's real batch data. Without it the app would return "looks fine" for a counterfeit box — worse than not checking at all, because it manufactures false confidence about medicine someone is about to swallow. Revisit only with a genuine data source (DGDA or a manufacturer feed).
6	Price comparison	Genuinely valuable for Bangladesh, but it needs current prices from many hospitals and labs. That is a data-collection and data-freshness problem, not a coding one, and stale prices make it useless. A good v4 once you have partners.
7	Mental health platform	Needs verified professionals, crisis-escalation paths and a duty-of-care policy. It is a product in its own right, and doing it badly in a corner of a hospital admin tool would be worse than not doing it.

8	Hospital bed / ICU availability	Only meaningful across many hospitals, each reporting live bed counts. Single-hospital MediDesk cannot produce the data. Strong candidate once multi-hospital support exists.
---	---------------------------------	---

On feature 2 specifically

I want to be direct because this one is different in kind. The other three exclusions are about effort and data. Medicine verification is about harm: a verification screen that cannot actually verify is not a neutral placeholder, it is a false assurance attached to a decision about someone's health. Build it when you have the data, not before.

3.3 Explicitly not doing

- **No AI or ML anywhere.** You asked for this and the whole document respects it — every feature is deterministic logic you can read and debug.
- No native mobile apps; the React app will be responsive instead.
- No payment gateway integration; billing continues to record payments, not process cards.
- No multi-hospital tenancy — which is precisely why features 6 and 8 are deferred.

4 Role-specific interfaces

The structural change. Everything in section 5 onward depends on this landing first.

Today MediDesk has **one interface** shared by everyone. Roles only change which sidebar links appear, which dashboard numbers load, and which rows the API returns. A doctor and a receptionist see the identical Appointments table, even though they need entirely different things from it: a doctor wants today's queue and a consult button, a receptionist wants to check people in and take payments.

That was the right call for an admin tool. It breaks down the moment patients are real users, because a patient should never see a staff table at all.

4.1 What each role gets

Role	Lands on	Their interface is built around
Patient	My Health	Next appointment with live queue position, my records, my reminders for today, my bills, donor status. No tables of other people. Mobile-first — most will be on a phone.
Doctor	Today's Clinic	A vertical list of today's patients in queue order, with the current patient pinned at the top, one click to open the consult view (history, allergies, prescribe, complete).
Receptionist	Front Desk	Check-in, queue management across all doctors, booking, and unpaid invoices. The operational console.
Pharmacist	Pharmacy	Prescriptions waiting to dispense, stock levels, expiring batches. Already largely exists; gets its own landing page.
Admin	Overview	The current dashboard, plus queue health across doctors and donor-network activity.

UI-1 · Split the shell by role

- Patients get a distinct layout — bottom navigation on mobile, no staff sidebar.
- Staff keep the sidebar, but the nav is generated per role rather than filtered from one list.
- Each role has its own landing route; login redirects there instead of everyone hitting `/dashboard`.
- Shared pieces stay shared — tables, forms, inputs — so there is one component library, not five.
- **Done when** logging in as each of the five roles lands somewhere purpose-built, and a patient can reach no staff screen by typing a URL.

UI-2 · Adopt React Query

- Every page currently hand-rolls `useEffect` fetching, with no caching, retry or invalidation. The queue makes that untenable — it needs polling and refetch-on-focus.
- Replace the fetching in all pages; mutations invalidate their queries.
- **Done when** no page component calls the API directly from `useEffect` .

5 Feature A — Smart Queue & Waiting Time

P0

EXTENDS EXISTING

The problem, concretely

A patient travels two hours to a clinic for a 10am appointment. The doctor is running late, or is not coming at all. Nobody tells them. They wait four hours in a corridor, guessing.

The fix is not clever scheduling — it is **honesty in real time**: where you are in the line, roughly how long that means, and whether the doctor is actually in.

5.1 How the queue works

A queue is not a new entity. It is an *ordering over appointments that have been checked in*, per doctor, per day. `Appointment.checked_in_at` already exists, which means position is derivable rather than stored:

```
# position = how many checked-in, not-yet-seen patients are ahead of you today
ahead = (Appointment.objects
    .filter(doctor=appt.doctor,
            appointment_date__date=today,
            checked_in_at__lt=appt.checked_in_at)
    .exclude(status__in=['completed', 'cancelled'])
    .count())

position = ahead + 1
estimated_wait = ahead * doctor.average_consult_minutes
```

`average_consult_minutes` starts at the doctor's `slot_duration_minutes` and is refined from that doctor's own recent completed consultations — a rolling average of the last 20. Plain arithmetic, no prediction model.

A-1 • Check-in creates a queue position

- Reception checks a patient in; `checked_in_at` is stamped and status becomes `checked_in`.
- A patient can self-check-in from their phone, but only within 30 minutes of their slot and only if marked as arrived by reception — self-check-in from home would poison the queue.
- `GET /api/v1/queue/me/` returns position, estimated wait and the doctor's status.
- **Done when** checking in three patients gives positions 1, 2, 3, and completing the first moves the others up.

A-2 · Live updates without refreshing

- Django Channels over WebSockets; each doctor's queue is a group, and any change broadcasts the new state to everyone watching.
- **Polling is the fallback, not an afterthought** — if the socket fails, React Query polls every 30 seconds and the user sees no difference. Ship polling first, add sockets second.
- The patient screen shows position, estimated wait, and a clear "you are next" state.
- **Done when** completing a consultation on the doctor's screen moves every waiting patient's position within two seconds, with no refresh.

A-3 · Honest doctor availability

- A doctor sets themselves *in clinic*, *on a break*, *running late* (with an estimate), or *unavailable today*.
- Status is visible before booking and while waiting. Marking unavailable notifies every patient booked that day, immediately.
- Search by specialisation and department, filtered to doctors actually available.
- **Done when** a doctor marking themselves unavailable results in a notification for every affected patient without any staff action.

A-4 · Notifications that reach people

- In-app notification always; email where present; SMS through a Bangladeshi gateway behind a provider interface so it can be swapped.
- Triggers: booking confirmed, doctor delayed or cancelled, you are third in line, you are next.
- All sending is a Celery task — a slow SMS gateway must never block a request.
- **Done when** SMS failure degrades to in-app only, with an error logged and no user-visible breakage.

Online consultation — deliberately minimal

You listed online consultation under this feature. v3.0 supports it as an **appointment type** (in-person or online) that carries a meeting link the doctor pastes in, plus a text consultation thread. Building video into the app is a large piece of work with its own infrastructure; a link to an existing tool gets 90% of the value for 5% of the cost.

6 Feature B — Digital Medical Records

P1

NEW

The problem, concretely

A patient sees a new doctor and is asked what they were prescribed last year. They think it was a white tablet. The report is in an envelope at home, or lost.

The patient owns the record. The hospital holds a copy. Sharing is explicit, time-limited and revocable — never a blanket grant.

B-1 · Upload and keep documents

- Upload prescriptions, test reports, scans and discharge summaries. PDF, JPG and PNG, maximum 10 MB.
- Each document has a type, a date it relates to (not just upload date), the issuing doctor or lab, and free-text notes.
- **Validate by content, not by file extension**, and serve uploads from a non-executable path. An upload endpoint is the most commonly exploited part of an app.
- Prescriptions written inside MediDesk appear in the timeline automatically — no upload needed.
- **Done when** a patient sees one chronological history combining uploaded documents and MediDesk-generated records.

B-2 · Sharing that the patient controls

- A patient grants a named doctor access to *selected* documents for a chosen window (default 7 days), and can revoke early at any time.
- Access is enforced in the queryset, so an expired or revoked share returns 404 like any other out-of-scope row.
- Every grant, view and revoke writes to the audit log.
- The patient can see exactly who currently has access and what they viewed.
- **Done when** a doctor's access disappears the moment it expires, proven by a test that travels past the expiry.

B-3 · Family profiles

- An adult can manage dependants — children, or elderly parents — under their own account.
- Dependants have records and reminders but no login of their own.
- A dependant reaching 18 can be converted to an independent account, taking their records with them.
- **Done when** a guardian can switch profiles and see only that dependant's data.

7 Feature C — Blood Donor Network

P1NEW

The problem, concretely

A patient needs two bags of O− at midnight. The family starts phoning everyone they know, and posting in Facebook groups, hoping someone matching is close enough to help.

The technical problem is easy — match blood group, filter by distance, notify. The real design problem is **protecting donors** from exposure and from being pestered.

7.1 Compatibility is a rule, not a lookup

Matching must use real transfusion compatibility, not string equality. An O− request should reach only O− donors, but an AB+ request can accept every group. This table is the specification:

Recipient needs	Can receive from
O−	O−
O+	O−, O+
A−	O−, A−
A+	O−, O+, A−, A+
B−	O−, B−
B+	O−, O+, B−, B+
AB−	O−, A−, B−, AB−
AB+	all groups

C-1 · Donor profiles and consent

- Registering as a donor is **opt-in and separate** from having a MediDesk account. Being a patient must never imply being a donor.
- Donor holds blood group, area, contact preference and availability toggle.
- **Automatic 90-day cooldown after a recorded donation**, during which the donor is not matched. This is a medical safety rule, not a convenience.
- Donors can go invisible or delete their donor profile entirely, without touching their patient account.
- **Done when** a donor who just donated stops appearing in searches for 90 days.

C-2 · Requests and matching

- A request carries blood group, units, hospital, urgency and a needed-by time.
- Matching applies the compatibility table above, then availability, then area, then cooldown.
- Notifications go out in waves — nearest area first, widening if nobody responds within 30 minutes — so a routine request does not alert the entire city at once.
- Donors respond yes or no; the requester sees a count of responders, never a donor list.
- **Done when** an AB+ request reaches every group and an O– request reaches only O– donors, both proven by tests.

C-3 · Contact details stay private until both agree

- **A donor's phone number is never in a list response.** It is revealed only after that donor accepts that specific request.
- Requests are rate-limited per user, and staff can verify a request as hospital-confirmed.
- Donors can report a request; repeated reports suspend the requester.
- Every reveal is audited.
- **Done when** no API response anywhere exposes a donor's contact details before that donor has accepted.

Location: keep it simple

Use **administrative areas** (district and thana), not GPS coordinates. A dropdown of areas gives you matching that is good enough, needs no PostGIS, no map bills, and no continuous tracking of where donors physically are. Precise location on a donor list is a safety problem, not a feature.

8 Feature D — Medicine Reminders

P2

EXTENDS EXISTING

The problem, concretely

An elderly patient is given four medicines with different schedules. Some doses are missed, some doubled. The family finds out weeks later, at the follow-up, when things have got worse.

D-1 · Schedules from prescriptions

- A prescription line already carries dosage, frequency and duration. Generate a schedule from it with one click rather than retyping.
- Patients can also add a medicine manually, for anything prescribed elsewhere.
- Each dose is a row with a due time and a taken/missed/skipped state.
- **Done when** a 7-day twice-daily prescription produces 14 dose rows at the right times.

D-2 · Reminders and adherence

- Celery Beat sweeps every 15 minutes and sends what is due — batched, never one task per dose.
- One tap marks a dose taken. Missing the window marks it missed automatically.
- A simple adherence percentage per medicine and overall, plus a follow-up reminder when a prescription has a follow-up date.
- **Done when** a dose left unconfirmed past its window is marked missed without anyone doing anything.

D-3 · Family escalation

- A patient nominates a family contact and chooses the trigger — for example three missed doses of the same medicine in a week.
- **The patient consents to this explicitly and can turn it off.** Reporting an adult's behaviour to a relative without their agreement is surveillance, however well-intentioned.
- For dependant profiles, the guardian is notified by default.
- **Done when** the third miss notifies the nominated contact, and disabling the setting stops it.

9 Data model changes

Fifteen models exist today. v3.0 adds eleven and extends four.

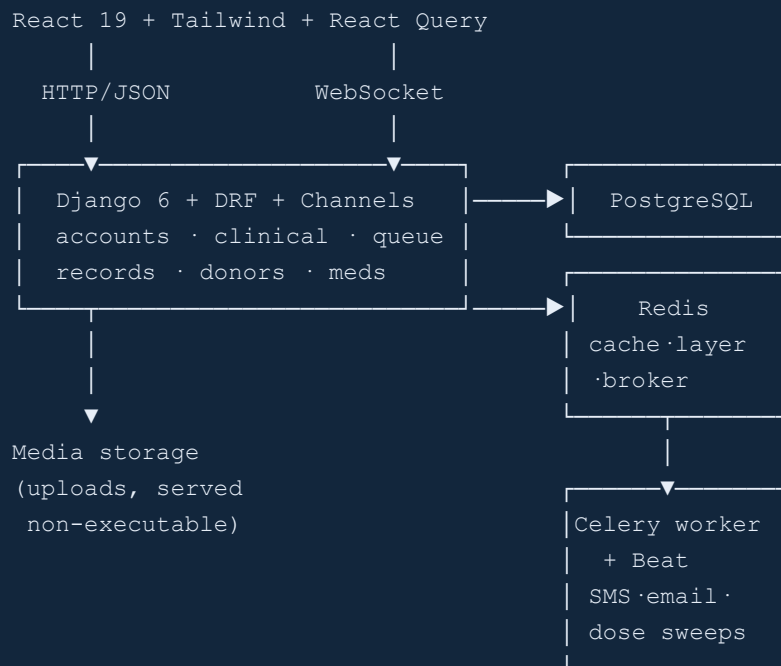
Model		Purpose and key fields
Doctor	extend	<code>availability_status</code> (in clinic / break / late / unavailable), <code>status_note</code> , <code>average_consult_minutes</code> .
Appointment	extend	<code>checked_in</code> and <code>in_consultation</code> added to status; <code>consultation_type</code> (in person / online), <code>meeting_link</code> , <code>started_at</code> .
Patient	extend	<code>guardian</code> self-FK for dependants, <code>district</code> , <code>thana</code> .
Notification	extend	<code>channel</code> (in-app / email / SMS), <code>delivery_status</code> , <code>category</code> .
QueueEntry	NEW	Optional denormalised snapshot — position, estimated wait, calculated-at. Add only if computing live proves slow; correctness first.
ConsultationNote	NEW	Text thread for online consultations, linked to an appointment.
MedicalDocument	NEW	Owner, type, file, document date, issuer, notes, size, checksum.
DocumentShare	NEW	Document, granted-to, granted-by, expires-at, revoked-at. Expiry enforced in the queryset.
DocumentAccessLog	NEW	Append-only record of every view, so the patient can see who read what.
Donor	NEW	User, blood group, district, thana, is-available, last-donation-date, contact preference.
BloodRequest	NEW	Requester, blood group, units, hospital, urgency, needed-by, status, verified-by.
DonorResponse	NEW	Request, donor, response, responded-at. Contact revealed only on acceptance.
DonationRecord	NEW	Donor, date, hospital, units. Drives the 90-day cooldown.
MedicationSchedule	NEW	Patient, medicine, dosage, times per day, specific times, start and end dates, active flag.

MedicationDose	NEW	Schedule, due-at, state, confirmed-at. One row per expected dose.
CareContact	NEW	Patient, name, phone, relationship, escalation threshold, consent flag.

9.1 New endpoints

Endpoint	Purpose
GET /queue/me/	My position, wait estimate, doctor status
GET /queue/doctor/{id}/	Full queue for staff and the doctor
POST /appointments/{id}/check-in/	Enter the queue
POST /appointments/{id}/start/ · /complete/	Advance the queue
PATCH /doctors/{id}/availability/	Set clinic status
ws://.../ws/queue/{doctor_id}/	Live queue updates
/documents/ · /documents/{id}/share/ · /shared-with-me/	Records vault
/donors/ · /blood-requests/ · /blood-requests/{id}/respond/	Donor network
/medication-schedules/ · /doses/{id}/taken/ · /adherence/	Reminders

10 Technical architecture



Addition	Why it is needed, specifically
Redis	Three jobs: Celery broker, cache for queue calculations, and the Channels layer that lets one worker broadcast to sockets held by another.
Celery + Beat	SMS and email must not block requests. Beat runs the dose sweep and the donor-request widening waves.
Django Channels	Live queue only. Everything else stays plain REST — do not spread WebSockets further than they earn their place.
React Query	Caching, refetch on focus, and polling as the queue's fallback transport.
PostgreSQL	Promote from SQLite. Concurrent check-ins and row locking during queue advancement need a real database.

Sequencing that keeps you sane

Build the queue with **polling first** and add WebSockets afterwards. Polling every 30 seconds is genuinely acceptable for a waiting room, it is a fraction of the work, and it becomes your fallback for free. Chasing sockets on day one is the most likely way to stall this project.

11 Non-functional requirements

Area	Requirement
Privacy	Donor contact details hidden until acceptance. Document shares expire by default. Every access to another person's record is audited. Uploads are private by default.
Security	Uploads validated by content type and size, stored outside the executable path. Rate limits on blood requests, document shares, check-in and OTP. The v2 scoping rules extend to every new endpoint.
Performance	Queue position under 200 ms; WebSocket broadcast under 2 s; donor matching under 1 s for 10,000 donors; dose sweep handles 10,000 patients within its 15-minute window.
Mobile	Patient screens are designed phone-first at 360 px and must be usable one-handed on a slow connection — this is how most patients will actually use it.
Reliability	SMS failure degrades to in-app. Socket failure degrades to polling. Celery being down delays notifications but never breaks a request.
Localisation	Bangla and English, with all user-facing strings translatable from the start — retrofitting i18n is far more painful than starting with it. Dates and phone formats local.
Testing	Backend coverage stays at 80% or better. The permission matrix grows to cover documents, donors and schedules. Blood compatibility and queue ordering get exhaustive tests.

12 Build order

Five phases. Each ends with something demonstrable.

Phase	Time	Contents and exit criteria
0 • Foundations	1 wk	PostgreSQL, Redis, Celery, React Query, role-specific shells (UI-1, UI-2). <i>Exit:</i> each role lands on its own page; no page fetches in <code>useEffect</code> .
1 • Queue, polled	1.5–2 wk	A-1, A-3, A-4 and the doctor and front-desk screens, refreshing by polling. <i>Exit:</i> a patient sees a position that changes as the doctor works through the list.
2 • Queue, live	1 wk	A-2 — Channels, with polling retained as fallback. Online consultation type. <i>Exit:</i> completing a consult updates every waiting screen within 2 s.
3 • Records	1.5–2 wk	B-1, B-2, B-3 — upload, timeline, expiring shares, family profiles. <i>Exit:</i> a share expires on time, proven by test; access log visible to the patient.
4 • Donors	1.5–2 wk	C-1, C-2, C-3 — compatibility matching, notification waves, privacy rules. <i>Exit:</i> no endpoint leaks contact details pre-acceptance; cooldown enforced.
5 • Reminders	1 wk	D-1, D-2, D-3 — schedules, Beat sweeps, adherence, consented escalation. <i>Exit:</i> a missed dose auto-marks and the third miss escalates.

Roughly 7–9 weeks solo. Phase 0 is genuinely load-bearing — the other phases assume the role-specific shells and React Query are already in place.

If you only ship one thing

Phases 0 to 2. A working live queue is the feature patients feel immediately, it is the hardest to copy, and it demonstrates real-time systems, background jobs and role-based UX in one coherent story. The other three features are additive; the queue is the product.

13 Risks

Risk	Impact	Mitigation
WebSockets stall the project	High	Polling ships first in phase 1 and remains the fallback. Phase 2 is then optional rather than blocking.

Wait estimates are wrong and lose trust	Med	Always present as a range with a caveat, never a countdown. Base it on that doctor's own recent history, and say plainly that it is an estimate.
Donor network is abused for spam or harassment	High	Rate limits, contact hidden until acceptance, reporting, staff verification, and the ability to go invisible instantly.
Upload endpoint becomes an attack vector	High	Content-type validation, size caps, non-executable storage path, and no user-controlled filenames on disk.
SMS costs or gateway reliability	Med	Provider behind an interface; in-app is always primary; SMS reserved for genuinely urgent events only.
Scope creep across four features	High	Phases are sequential and each ends demonstrable. Finish a phase before starting the next.

14 Appendix

14.1 Dependencies to add

Backend

- `channels` + `channels-redis` — WebSockets
- `celery` + `django-celery-beat`
- `redis` + `django-redis`
- `psycopg[binary]` — PostgreSQL
- `pillow` — image handling
- `python-magic` — real content-type checks
- `daphne` — ASGI server

Frontend

- `@tanstack/react-query`
- `react-hook-form` + `zod`
- `date-fns`
- `recharts` — adherence charts
- `sonner` — toasts

Services

- A Bangladeshi SMS gateway
- SMTP for email

14.2 Definition of done

- ☐ Acceptance criteria demonstrably met.
- ☐ Tests written; coverage has not dropped.
- ☐ Every new endpoint has a row in the permission matrix test.
- ☐ Migrations included and reversible.
- ☐ `ruff` and `eslint` clean; `check --deploy` still zero issues.
- ☐ Patient-facing screens verified at 360 px width.
- ☐ No personal data exposed that the feature does not strictly require.
- ☐ User-facing strings are translatable.

14.3 What success looks like

Measure	Target
Patient knows their queue position	Without asking staff, on their own phone
Wait estimate accuracy	Within 15 minutes for 80% of visits
Blood request first response	Under 10 minutes for urgent requests
Donor contact leaked before acceptance	Zero — enforced by test

Expired share still readable	Zero — enforced by test
Medication adherence, reminded patients	Above 85%
Backend coverage	80% or better, 100% on permissions